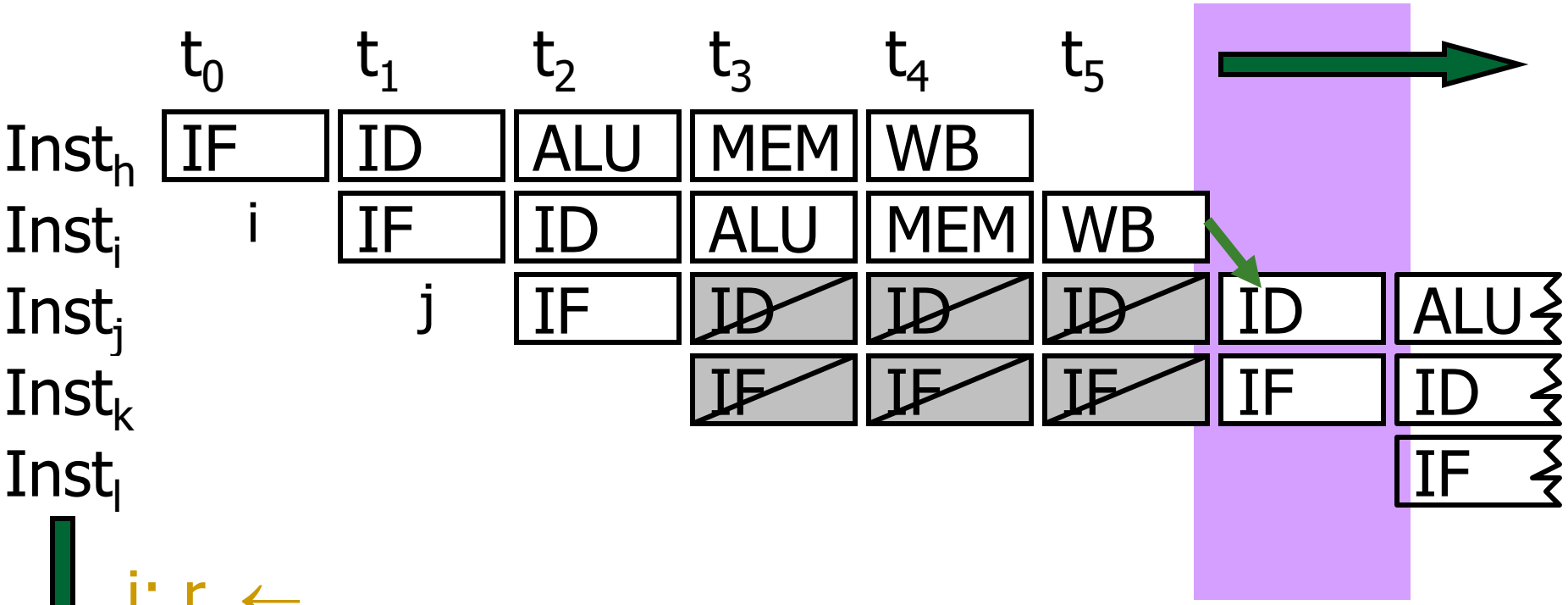
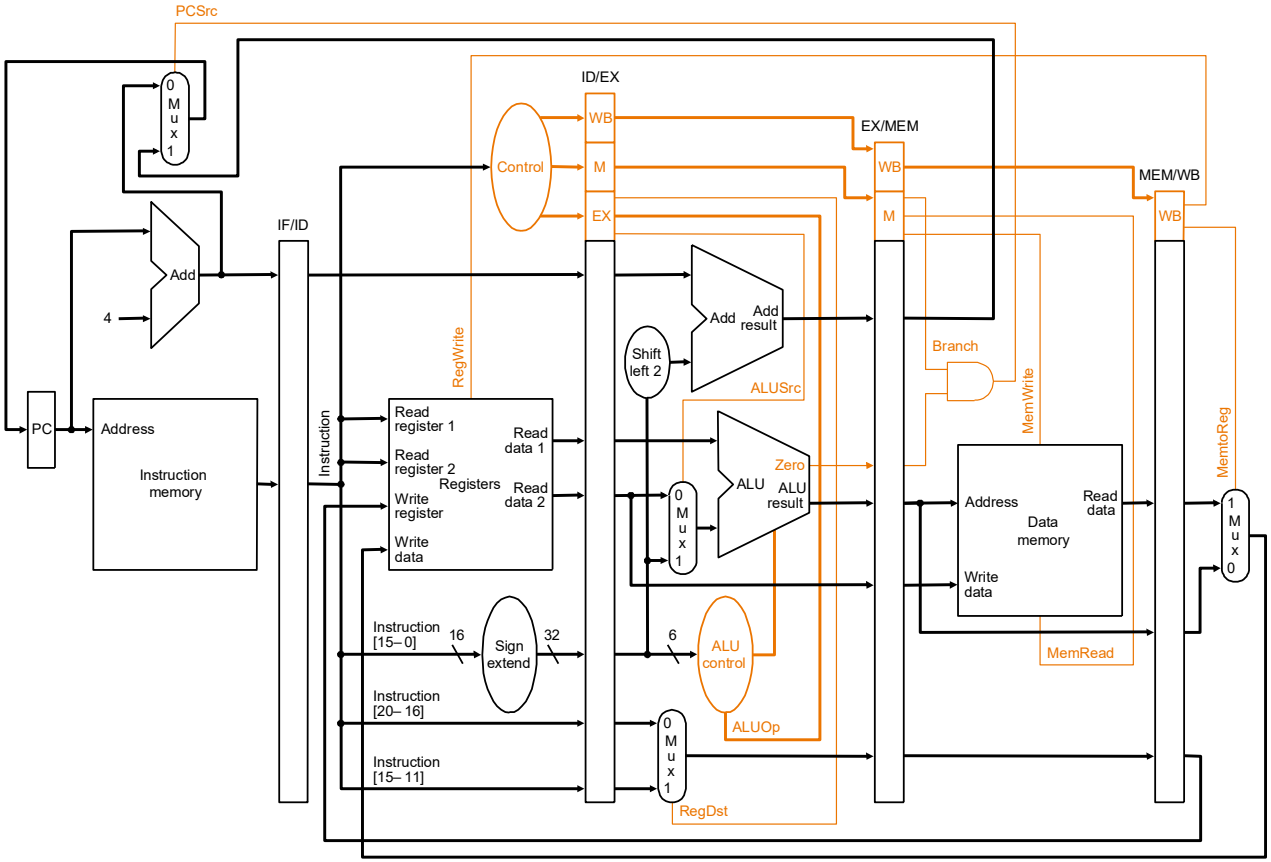
回顾：流水线停顿：解决数据依赖



i: $r_x \leftarrow$ _
bubble
bubble
bubble
j: _ $\leftarrow r_x$ dist(i,j)=4

停顿（Stall）：使依赖的指令等待，直到其源数据值可用。
执行步骤：
停止所有上游阶段（up-stream stages）
排空所有下游阶段（down-stream stages）

回顾：如何实现流水线停顿（Stalling）



- **停顿（Stall）机制：**
  - 禁用 PC 和 IR 锁存（latching）；确保发生停顿的指令停留在其当前阶段（stage）中，不会前进。
  - 插入 “无效” 指令 或 空操作（NOP）；将这些指令插入到紧随停顿指令之后的阶段中，这些空操作被称为 “气泡（bubbles）”。

回顾：停顿条件（Stall Conditions）

- 指令 I_a 和 I_β（其中 I_a 在 I_β 之前）具有 RAW（读后写）依赖 当且仅当：
  - I_β（R/I、LW、SW、Br 或 JR）读取了 I_a（R/I 或 LW）写入的寄存器
  - $dist(I_a, I_\beta) \leqslant dist(ID, WB) = 3$

- 当 I_β 在 ID 阶段 想要读取一个寄存器，而该寄存器将由 I_a 在 EX、MEM 或 WB 阶段 写入时，必须 让 ID 阶段停顿（stall）。

回顾：停顿（Stall）对性能的影响

- 每一个停顿周期（stall cycle）都对应于 一个丢失的周期（one lost cycle），
  在这个周期中，没有任何指令可以完成。

- 对于一个包含 N 条指令 且有 S 个停顿周期 的程序：
  平均 CPI（每指令周期数） = (N + S) / N

- S 的大小取决于：
  - RAW（读后写）依赖的频率
  - 依赖指令之间的具体距离
  - 依赖之间的分布
    - 比如若 i_1、i_2 和 i_3 都依赖于 i_0，一旦 i_1 的依赖被解决，则 i_2 和 i_3 通常也能顺利执行。

汇编例子

```
            addi    $s1, $s0, -1
for2tst:    slti    $t0, $s1, 0
            bne     $t0, $zero, exit2
                        .........
            addi    $s1, $s1, -1
            j       for2tst
exit2:
```

3 stalls

3 stalls

通过数据前递减少停顿

- 也叫做数据旁路（Data Bypassing）

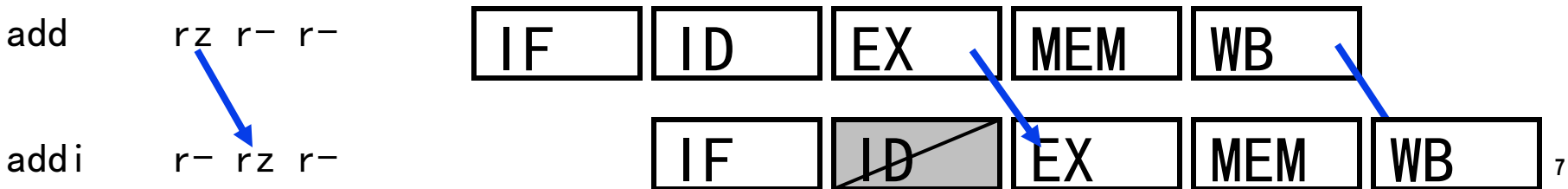
- 我们之前已经看过了基本概念
- 尽快将数据传递给依赖的指令，
  一旦数据可用

- 还记得数据流吗？
  - 数据值在可用时立刻提供给依赖的指令
  - 指令在所有操作数可用时执行

- 数据前递使得流水线更接近数据流执行原理。

数据前递（或数据旁路）

- **Data Forwarding (or Data Bypassing) 直观的理解：**
  - 将寄存器文件 (RF) 理解为状态：
    - "add rx ry rz" 的字面意思是从 RF[ry] 和 RF[rz] 获取值，并将结果放入 RF[rx]。
- **但，寄存器文件仅是通信抽象的一部分：**
  - "add rx ry rz" 的意思是：
    - 获取最后几条指令的结果来定义 RF[ry] 和 RF[rz] 的值。
    - 直到另一条指令重新定义 RF[rx]，后续引用 RF[rx] 的指令应使用该指令的结果。
- **关键是保持操作之间的正确“数据流”，因此：**
  - 通过数据路径而非寄存器文件 (RF) 获取操作数。
  - 如果多个定义还在进行，获取最新的定义。

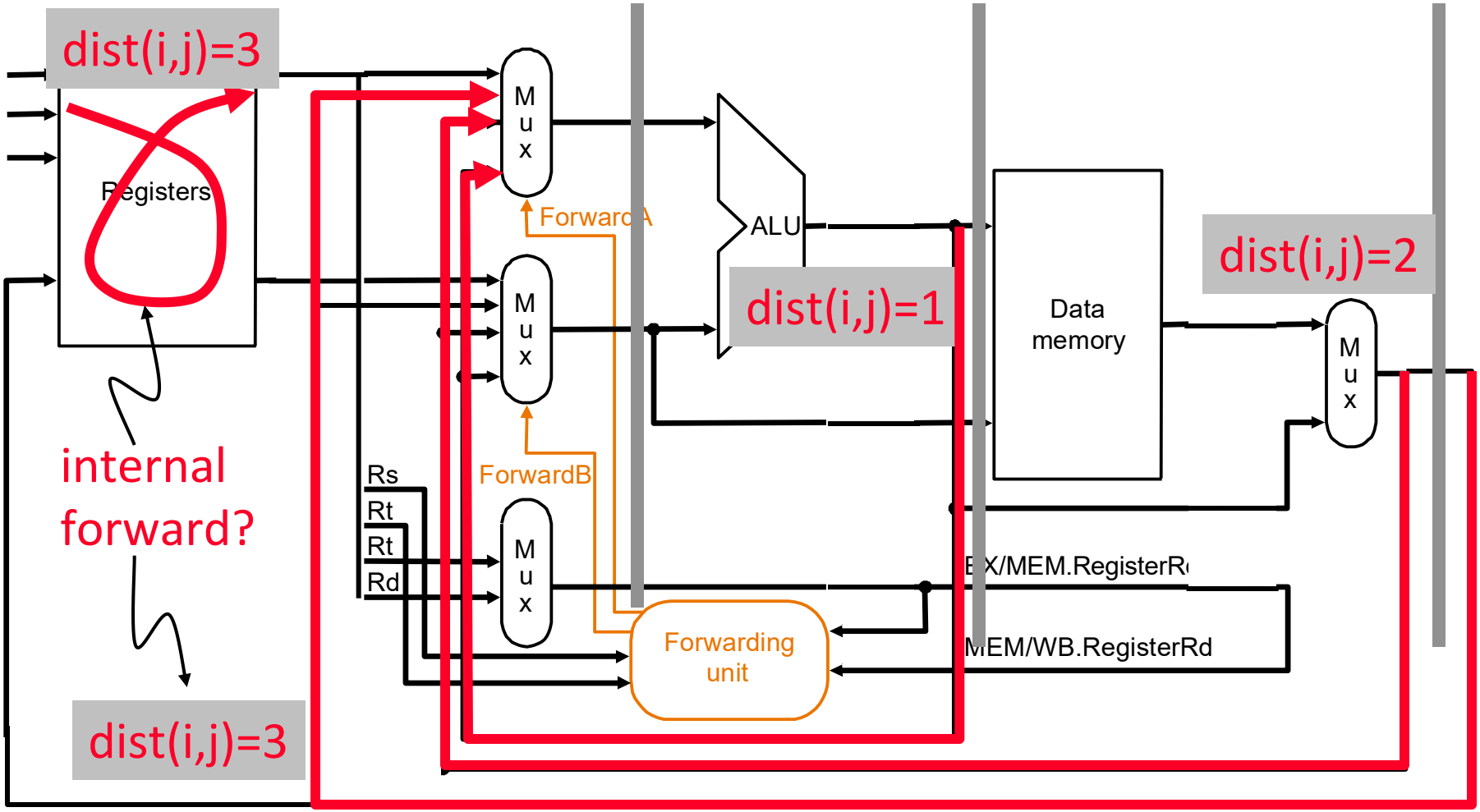


使用数据前递解决 RAW 依赖

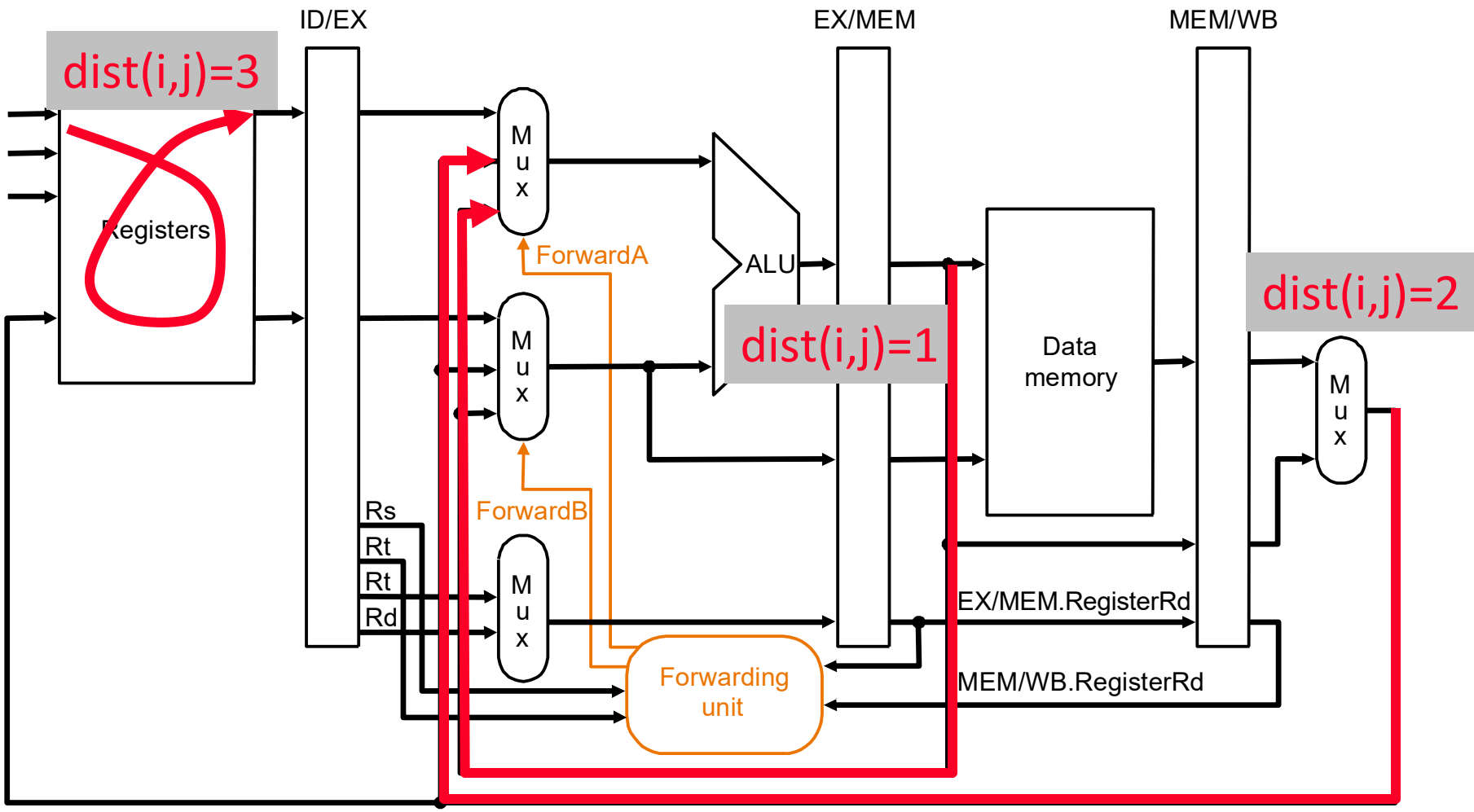
- 指令 IA 和 IB（其中 IA 在 IB 之前执行）具有 RAW 依赖，当且仅当：
  - IB（R/I 类型，LW，SW，Br 或 JR）读取由 IA（R/I 或 LW）写入的寄存器。
  - dist(IA, IB) ⩽ dist(ID, WB) = 3。

- 换句话说，如果 IB 在 ID 阶段读取 IA 在 EX、MEM 或 WB 阶段写入的寄存器，那么 IB 需要的操作数尚未在寄存器文件（RF）中：
  - 从数据路径中获取操作数，而不是从寄存器文件（RF）。
  - 如果有多个定义，获取最新定义的操作数。

数据前递 Data Forwarding Paths （v1）



数据前递 Data Forwarding Paths (v2)



数据前递逻辑（针对v2）

```
if (rSEx != 0) && (rSEX == destMEM) && RegWriteMEM then
   从MEM阶段前递操作数  // dist=1
else if (rSEx != 0) && (rSEX == destWB) && RegWriteWB then
   从WB阶段前递操作数  // dist=2
else
   使用寄存器文件中的操作数  // dist >= 3
```

- 排序很重要！必须首先检查最年轻的匹配项。

数据前递（依赖分析）

LW $1, 0($2)

	R/I-Type	LW	SW	Br	J	Jr
IF						
ID						use
EX	use produce	use	use	use		
MEM		produce	(use)			
WB						

- use：表示某条指令在该阶段需要使用某个数据（寄存器的值或内存的值）。例如，如果指令需要读取寄存器或内存中的值以进行计算，这时我们称该指令在当前阶段“使用”数据。
- produce：表示某条指令在该阶段会产生某个数据（例如计算结果或数据存储到寄存器或内存中）。例如，在执行加法运算的指令中，计算结果会在执行阶段结束时“生成”或“产生”并存储在目标寄存器中。
- 即使有数据前递，紧接在LW指令前的RAW依赖仍然需要停顿

示例汇编，无数据前递

- for (j=i-1; j>=0 && v[j] > v[j+1]; j-=1) { ..... }

```
                addi    $s1, $s0, -1
for2tst:        slti    $t0, $s1, 0
                bne     $t0, $zero, exit2
                sll     $t1, $s1, 2
                add     $t2, $a0, $t1
                lw      $t3, 0($t2)
                lw      $t4, 4($t2)
                slt     $t0, $t4, $t3
                beq     $t0, $zero, exit2
                ……
                addi    $s1, $s1, -1
                j       for2tst
exit2:
```

3 stalls
3 stalls
3 stalls
3 stalls
3 stalls
3 stalls

示例汇编 （Revisited）

- **for (j=i-1; j>=0 && v[j] > v[j+1]; j-=1) { …… }**

```
                addi    $s1, $s0, -1
for2tst:        slti    $t0, $s1, 0
                bne     $t0, $zero, exit2
                sll     $t1, $s1, 2
                add     $t2, $a0, $t1
                lw      $t3, 0($t2)
                lw      $t4, 4($t2)
                nop
                slt     $t0, $t4, $t3
                beq     $t0, $zero, exit2
                ……
                addi    $s1, $s1, -1
                j       for2tst
exit2:
```

思考

- **硬件与软件在数据依赖处理中的角色是什么？**
  - 基于软件的互锁
  - 基于硬件的互锁
  - 谁插入/管理流水线中的气泡（无操作指令）？
  - 谁找到独立的指令来填充“空白”流水线槽？
  - 每种方法的优缺点是什么？

思考

- **硬件与软件在指令执行顺序中的角色是什么？**
  - 基于软件的指令调度 → 静态调度
  - 基于硬件的指令调度 → 动态调度

更多关于软件与硬件的信息

- **基于软件的指令调度 → 静态调度**
  - 编译器安排指令顺序，硬件按此顺序执行
  - 与动态调度对比（在动态调度中，硬件可以按顺序外的顺序执行指令）
  - 编译器如何知道每条指令的延迟？
- **编译器不知道哪些信息，使得静态调度变得困难？**
  - **答案：** 在运行时确定的任何内容
    - 例如：**操作延迟的可变长度，内存地址，分支方向**
- **编译器如何缓解这一问题（即估算未知内容）？**
  - **答案：** 使用性能分析（Profiling）

今天的内容

- 控制依赖处理-续

回顾

- **数据依赖处理**
  - 数据转发/绕过
  - 深入实现
    - 寄存器依赖性分析
    - 阻塞
    - 有无转发的性能分析
  - **思考问题**
    - 硬件与软件在数据依赖处理上的区别
    - 静态调度与动态调度
    - 为什么基于编译器的指令调度很难？

回顾：控制依赖

- 问题：下一个周期中，取指令的PC应该是多少？
  - 答案：下一个指令的地址
  - 所有指令都依赖于前面的指令，为什么？
- 如果取出的指令是非控制流指令：
  - 下一个取指令的PC是下一条顺序指令的地址
  - 如果知道取出指令的大小，很容易确定
- 如果取出的指令是控制流指令：
  - 我们如何确定下一个取指令的PC？
- 实际上，我们如何知道取出的指令是否是控制流指令？

分支类型

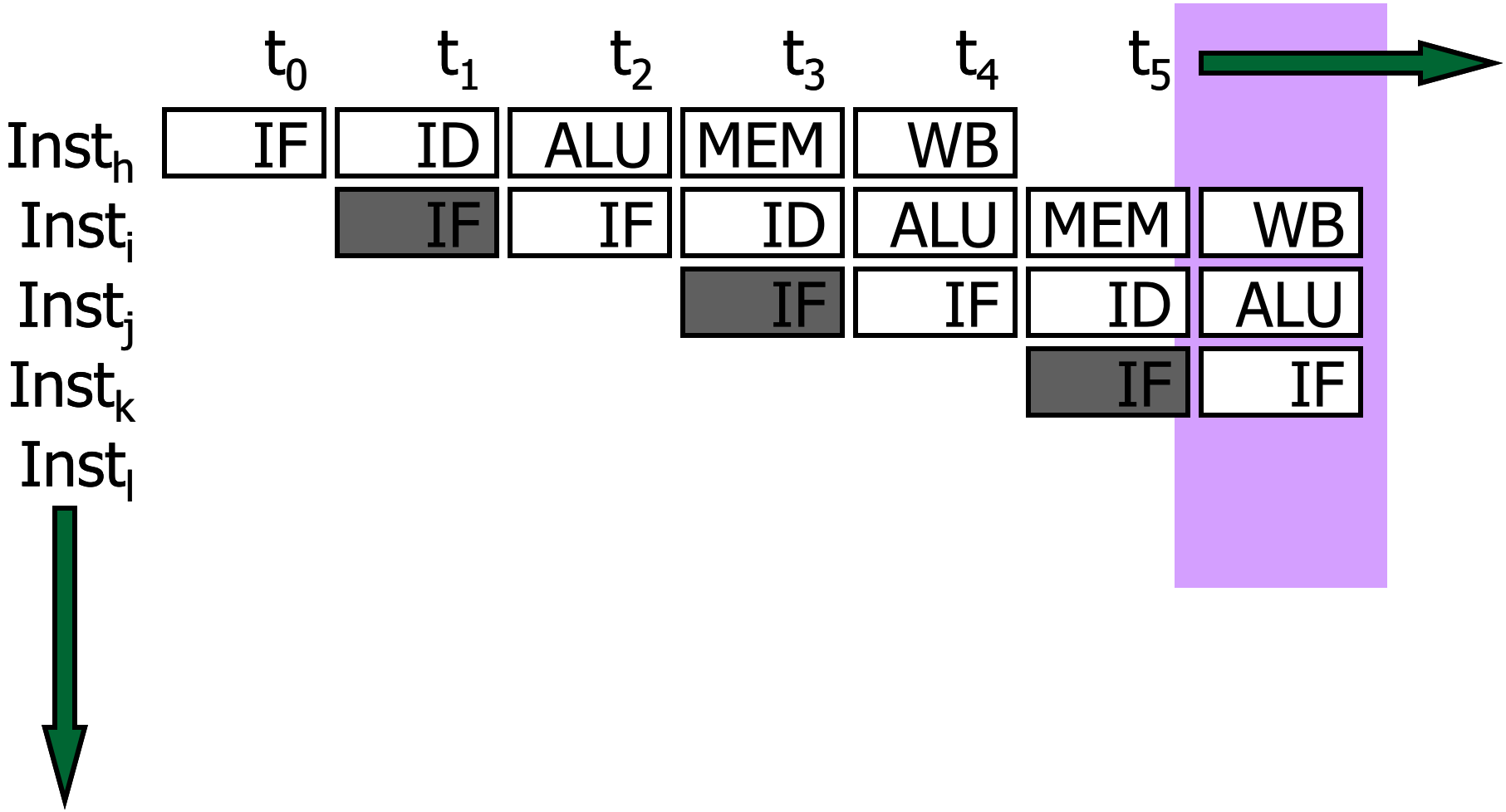
Type	Direction at fetch time	Number of possible next fetch addresses?	When is next fetch address resolved?
Conditional	Unknown	2	Execution (register dependent)
Unconditional	Always taken	1	Decode (PC + offset)
Call	Always taken	1	Decode (PC + offset)
Return	Always taken	Many	Execution (register dependent)
Indirect	Always taken	Many	Execution (register dependent)

不同的分支类型可以采用不同的处理方式

如何处理控制依赖

- 保持流水线完整，确保动态指令的正确顺序是至关重要的。
- 如果指令是控制流指令，潜在的解决方案有：
- 暂停流水线，直到我们知道下一个取址地址。
- 猜测下一个取址地址（分支预测）。
- 使用延迟分支（分支延迟槽）。
- 做其他处理（细粒度多线程）。
- 消除控制流指令（谓词执行 predicated execution）。
- 从两个可能的路径中取指（如果你知道两个路径的地址）（多路径执行）。

暂停取指直到下一个PC地址可用？



这是非控制流和无条件分支（br）指令的情况！

比等待取指停顿更好的方法…

- 与其等待PC的真依赖解决，不如猜测下一个PC = 当前PC + 4，继续每个周期取指。
  这是一个好的猜测吗？
  如果猜错了，你会失去什么？

- 约20%的指令是控制流指令
  - 其中大约50%的“前向”控制流（即if-then-else）会被执行
  - 大约90%的“后向”控制流（即循环回退）会被执行
    总体上，通常约70%会被执行，30%不会被执行 [Lee and Smith, 1984]

- 预计“nextPC = PC + 4”大约86%的时间，但剩余的14%呢？

猜测下一条指令地址 = 当前PC + 4

- 始终预测下一条顺序执行的指令是接下来要执行的指令
- 这是一种“下一个取指地址预测”（以及分支预测）的方法
- 如何让这个预测更有效？
- 想法：最大化下一个顺序指令是接下来要执行的指令的机会
  - 软件：布置控制流图，使得“可能的下一条指令”位于分支的未取路径上
    - 使用配置文件指导的代码定位 → Pettis & Hansen, PLDI 1990
  - 硬件：???（如何在硬件中做到这一点……）
    - 执行指令的追踪 → 追踪缓存

猜测下一条指令地址 = 当前PC + 4

- 如何使这一方法更加有效？
- 想法：去除控制流指令（或最小化它们的出现）
- 如何做到？
  1. 去除不必要的控制流指令 → 合并谓词（谓词合并）combine predicates (predicate combining)
  2. 将控制依赖转换为数据依赖 → 谓词执行 predicated execution
- 在计算机科学和编程中，谓词（Predicate）是一个函数或操作，它接受一个或多个输入参数并返回一个布尔值（true 或 false）。谓词通常用于条件判断、决策结构或控制流中，确定程序是否应该继续执行某个操作。

谓词：

- 条件判断：谓词（Predicate）广泛用于控制流语句中，例如 if 和 while。

```
if is_positive(x):
    print("x is positive")
else:
    print("x is not positive")
```

- 这里的 is_positive(x) 就是一个谓词，决定了 if 语句的执行路径。
- 函数式编程：在函数式编程中，谓词常常用来过滤集合或列表，例如：

```
numbers = [1, -2, 3, -4, 5]
positive_numbers = filter(is_positive, numbers)
print(list(positive_numbers))  # 输出 [1, 3, 5]
```

谓词合并（而非谓词执行）

- **复杂的谓词(Predicates)会被转换成多个分支：**
  - 例如，如果((a == b) && (c < d) && (a > 5000)) { … }
  - 这会有3个条件分支
- **问题：这会增加控制依赖的数量**
- **想法：合并谓词操作，将结果传递给一个单一的分支指令，而不是为每个谓词生成一个分支**
  - 谓词通过条件寄存器进行存储和操作
  - 单个分支指令检查合并后谓词的值
  - 代码中分支较少 → 较少的误预测/停顿
  - 可能的多余工作
  - 如果第一个谓词为假，则无需计算其他谓词
- **条件寄存器存在于IBM RS6000和POWER架构中**

如何处理控制依赖

- 保持流水线完整，确保动态指令的正确顺序是至关重要的。

- 如果指令是控制流指令，潜在的解决方案有：

- 暂停流水线，直到我们知道下一个取址地址。
- 猜测下一个取址地址（分支预测）。
- 使用延迟分支（分支延迟槽）。
- 做其他处理（细粒度多线程）。
- 消除控制流指令（谓词执行）。
- 从两个可能的路径中取指（如果你知道两个路径的地址）（多路径执行）。

谓词执行 Predicated Execution

- 概念：将控制依赖转换为数据依赖

- 简单示例：假设我们有一个条件移动指令...
  - CMOV condition, R1 ← R2
  - R1 = (condition == true) ? R2 : R1
  - 在大多数现代指令集架构中都有应用（如 x86、Alpha）

- 带分支与 CMOV 的代码示例：
  - if (a == 5) {b = 4;} else {b = 3;}

```
CMPEQ condition, a, 5;
CMOV condition, b ← 4;
CMOV !condition, b ← 3;
```

谓词执行 Predicated Execution

- 消除分支 → 启用直线代码（即代码中的更大基本块）
- 优点
  - 始终不取的预测效果更好（没有分支）
  - 编译器有更多的优化代码自由度（没有分支）
    - 控制流不会阻碍指令重排优化
    - 代码优化仅受数据依赖影响
- 缺点
  - 无效工作：某些指令被获取/执行但被丢弃（尤其是对于容易预测的分支来说尤其糟糕）
  - 需要额外的ISA支持
- 我们能以这种方式消除所有分支吗？

如何处理控制依赖

- 保持流水线完整，确保动态指令的正确顺序是至关重要的。

- 如果指令是控制流指令，潜在的解决方案有：

- 暂停流水线，直到我们知道下一个取址地址。
- 猜测下一个取址地址（分支预测）。
- 使用延迟分支（分支延迟槽）。
- 做其他处理（细粒度多线程）。
- 消除控制流指令（谓词执行）。
- 从两个可能的路径中取指（如果你知道两个路径的地址）（多路径执行）。

延迟分支（I）

- 改变分支指令的语义
  - 在 N 条指令后分支
  - 在 N 个周期后分支

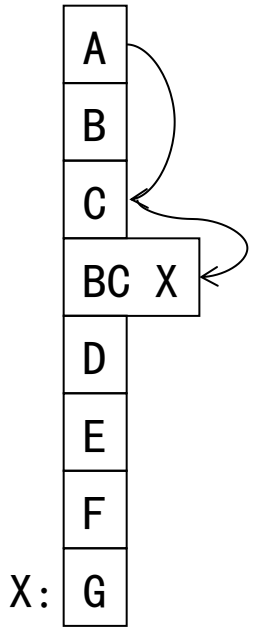
- 概念：延迟执行分支。分支后面的 N 条指令（延迟槽）无论分支方向如何，始终执行。

- 问题：如何找到填充延迟槽的指令？
  - 分支必须独立于延迟槽指令

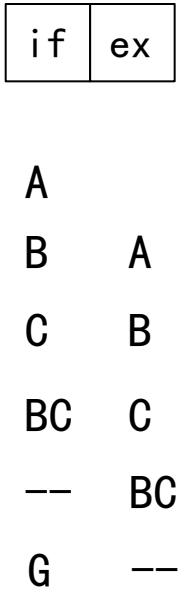
- 无条件分支：更容易找到指令来填充延迟槽
- 条件分支：条件计算不应依赖于延迟槽中的指令 → 很难填充延迟槽

延迟分支（II）

Normal code:

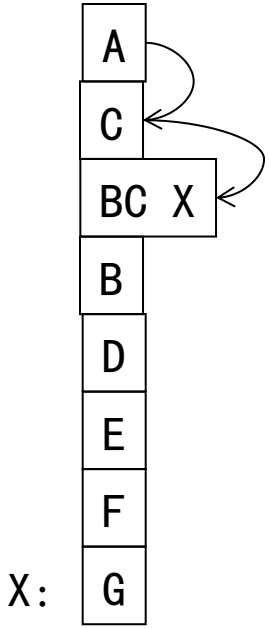


Timeline:

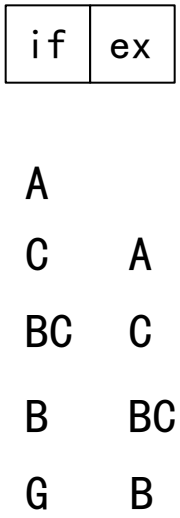


6 cycles

Delayed branch code:



Timeline:

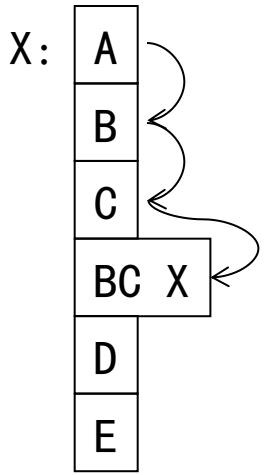


5 cycles

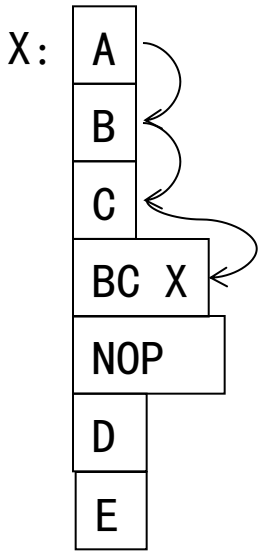
高级延迟分支（III）

- 带有冲刷的延迟分支（Delayed branch with squashing）
  - 在 SPARC 中
  - 语义：如果分支没有执行（即未被取走），则延迟槽指令不执行
  - 为什么这样会有帮助？

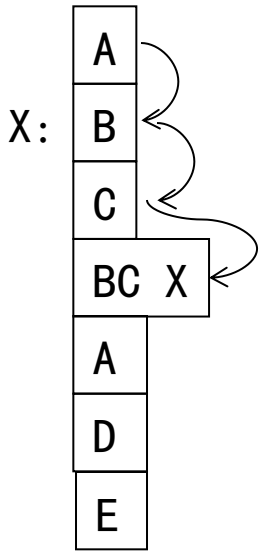
Normal code:



Delayed branch code:



Delayed branch w/ squashing:



延迟分支 (IV)

- **优点：**
  - 通过假设分支解决前，延迟槽数量等于保持流水线满的指令数量，简单地保持流水线满
  - 延迟槽的数量 == 需要的指令数量，以保持流水线在分支解决前满
  - 所有延迟槽都可以用有用的指令填充
- **缺点：**

  -- 即使在 2 阶流水线中，也不容易填充延迟槽
  - 延迟槽的数量随着流水线深度、超标量执行宽度的增加而增加
  - 延迟槽的数量应该随着不同延迟操作而变化。为什么？
  - -- 将 ISA 语义与硬件实现绑定

    -- SPARC、MIPS、HP-PA：1 个延迟槽

    -- 如果流水线实现发生变化，下一代设计会怎样？

如何处理控制依赖

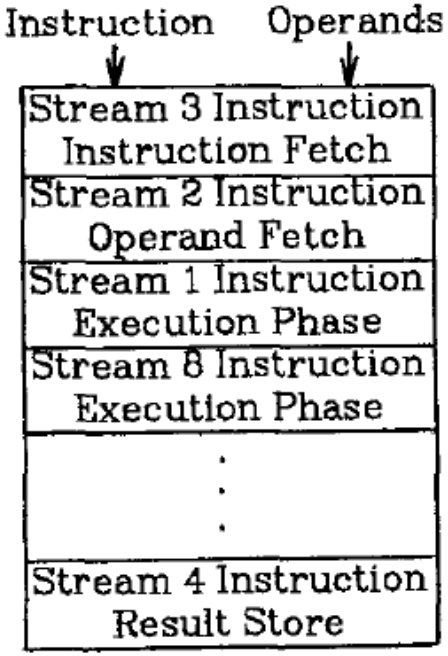
- 保持流水线完整，确保动态指令的正确顺序是至关重要的。

- 如果指令是控制流指令，潜在的解决方案有：

- 暂停流水线，直到我们知道下一个取址地址。
- 猜测下一个取址地址（分支预测）。
- 使用延迟分支（分支延迟槽）。
- 做其他处理（细粒度多线程）。
- 消除控制流指令（谓词执行）。
- 从两个可能的路径中取指（如果你知道两个路径的地址）（多路径执行）。

细粒度多线程

- 概念：硬件具有多个线程上下文。每个周期，取指引擎从不同的线程中取指。
  - 当取到的分支/指令解决时，来自同一线程的指令未被取出
  - 分支/指令解决的延迟与其他线程指令的执行重叠
- 优点：
  - 不需要处理线程内的控制和数据依赖
  - 不同线程的执行不会互相干扰
- 缺点：
  - 单线程性能下降
  - 需要额外的逻辑来保持线程上下文
  - 如果线程不足，无法覆盖整个流水线，则不能重叠延迟



细粒度多线程：历史

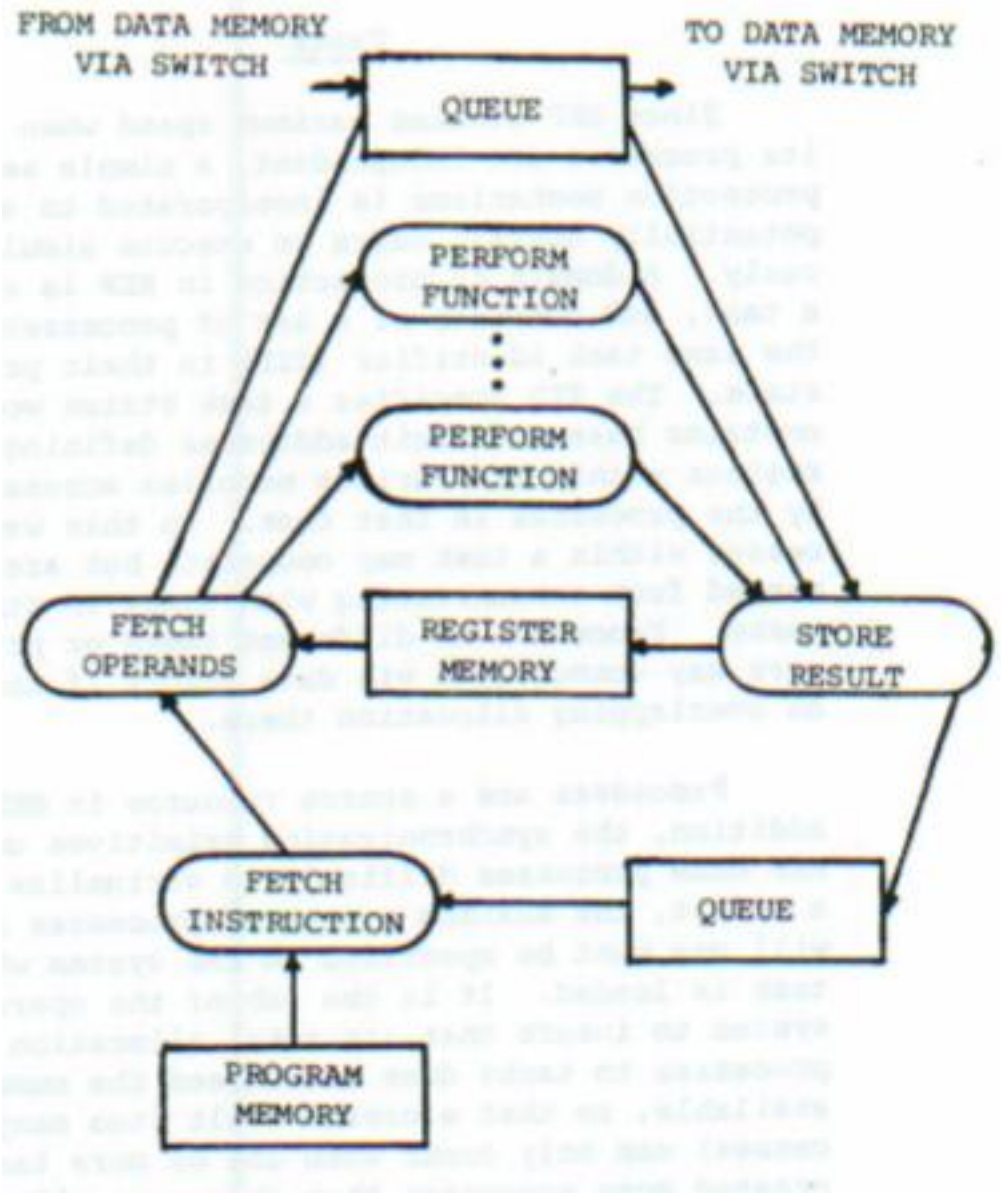
- CDC 6600的外围处理单元是细粒度多线程的
  - Thornton, “Parallel Operation in the Control Data 6600,” AFIPS 1964.
  - 处理器每个周期执行一个不同的I/O线程
  - 同一线程的操作每10个周期执行一次

- Denelcor HEP（异构元素处理器）
  - Smith, “A pipelined, shared resource MIMD computer,” ICPP 1978.
  - 每个处理器120个线程
  - 可用队列与不可用（等待）队列的线程
  - 每个线程在处理器流水线中只能有一条指令；每个线程是独立的
  - 对每个线程来说，处理器看起来像一个非流水线机器
  - 系统吞吐量与单线程性能之间的权衡

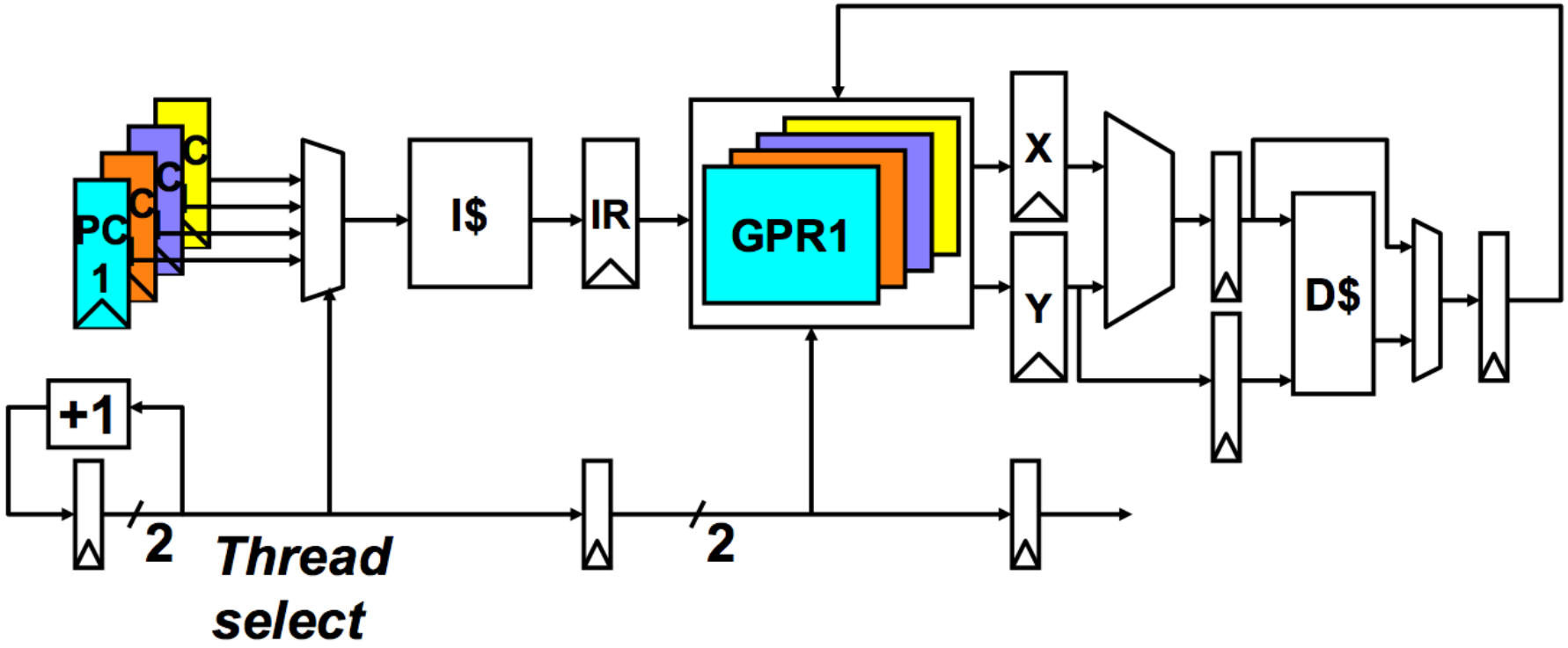
HEP中的细粒度多线程

- 周期时间：100ns
- 8个阶段 → 完成一条指令需要800ns
  - 假设没有内存访问
- 没有控制和数据依赖检查

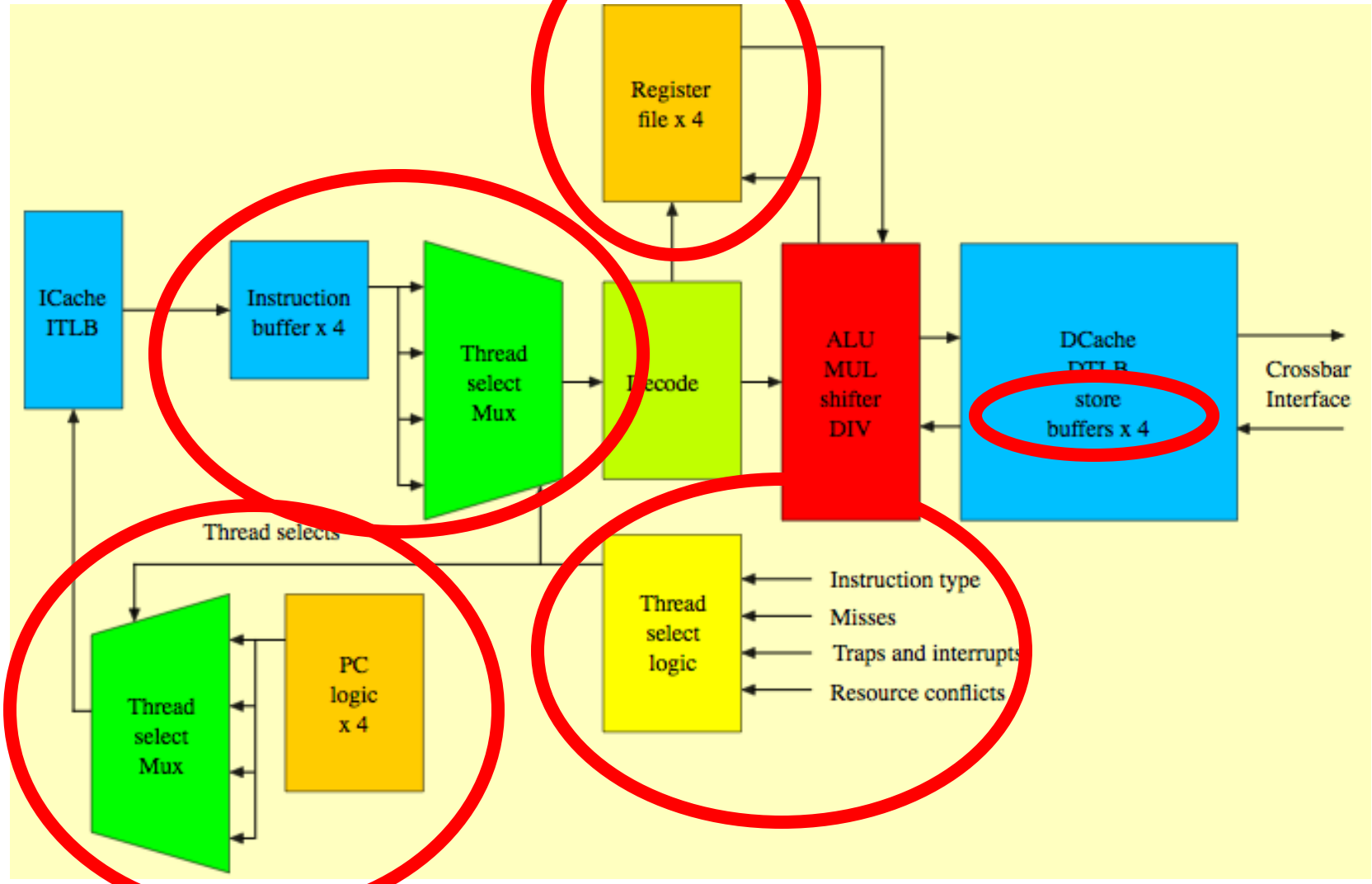




多线程流水线示例



Sun Niagara多线程流水线示例



Kongetira et al., "Niagara: A 32-Way Multithreaded Sparc Processor," IEEE Micro 2005.

细粒度多线程

- 优点
  - 不需要指令间的依赖性检查
    （每个周期只有一个线程的指令在流水线中）
  - 不需要分支预测逻辑
  - 其他“气泡”周期用于执行来自不同线程的有用指令
  - 提高系统吞吐量、延迟容忍度和利用率
- 缺点
  - 额外的硬件复杂性：多个硬件上下文（程序计数器、寄存器文件等），线程选择逻辑
  - 降低单线程性能（每隔 N 个周期从同一线程取一条指令）
  - 线程之间在缓存和内存中的资源竞争
  - 线程间依赖性检查逻辑仍然存在（加载/存储）

如何处理控制依赖

- 保持流水线完整，确保动态指令的正确顺序是至关重要的。
- 如果指令是控制流指令，潜在的解决方案有：
- 暂停流水线，直到我们知道下一个取址地址。
- 猜测下一个取址地址（分支预测）。
- 使用延迟分支（分支延迟槽）。
- 做其他处理（细粒度多线程）。
- 消除控制流指令（谓词执行）。
- 从两个可能的路径中取指（如果你知道两个路径的地址）（多路径执行）。